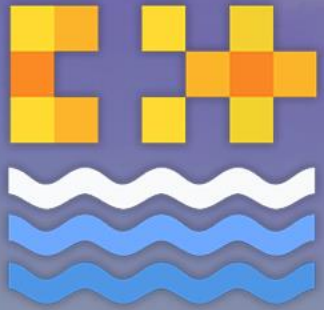




An Introduction to Swarm Intelligence Algorithms

(Swarm Your Way out of a Paper Bag)

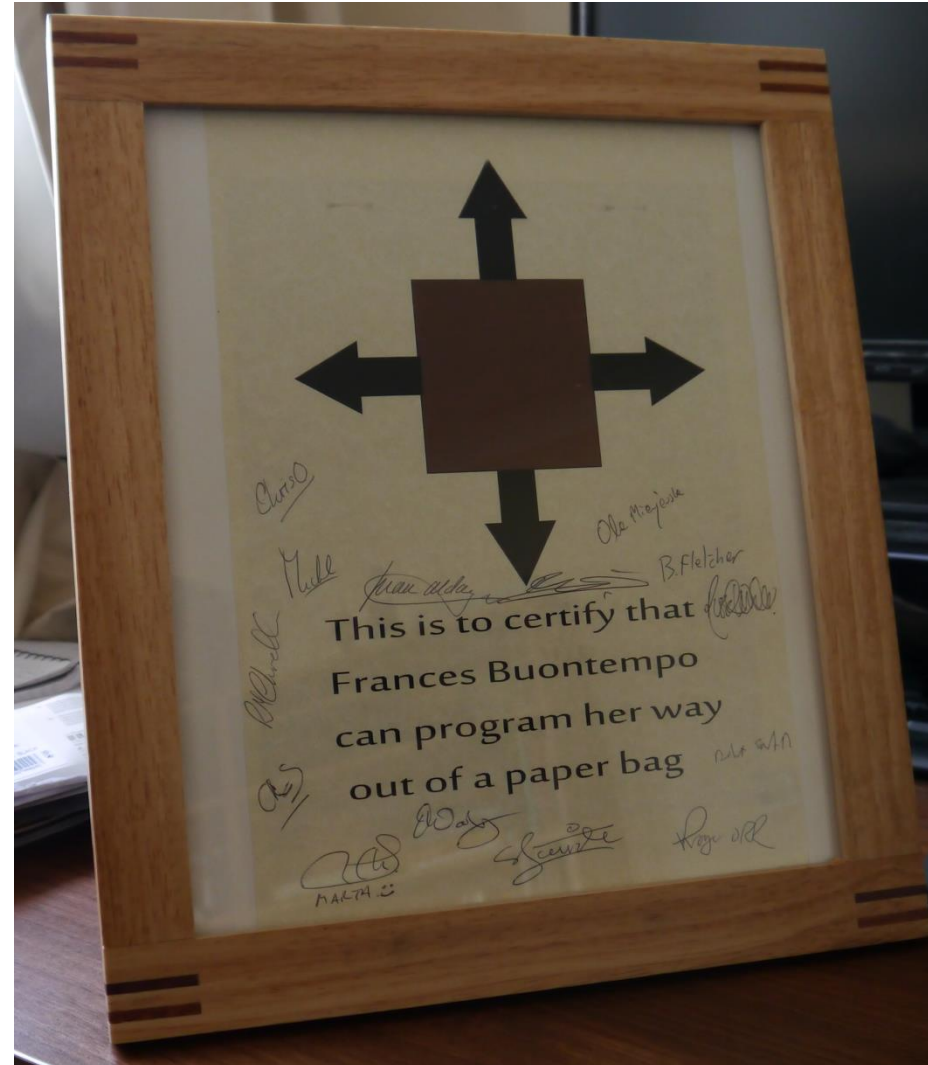
Frances Buontempo

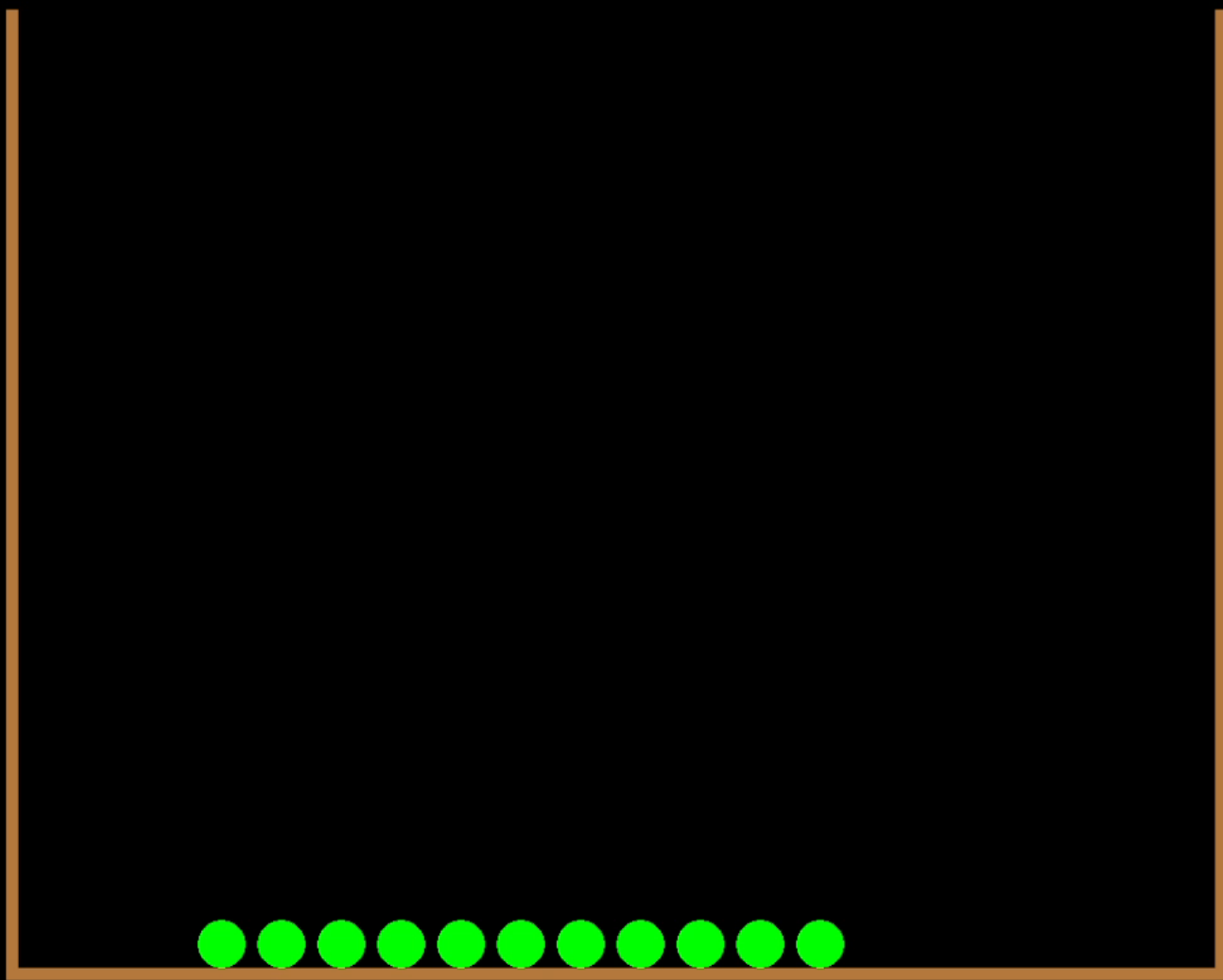
2024

Outline

- What is swarm intelligence?
- An overview of "nature inspired" AI algos and how to implement a few
- Particles, ants, bees, cats....
 - Yes! Herding cats!!
- Some demos of "cats" escaping a paper bag, as a simple way to visualise how the swarms move

Award!





Simple and Fast Media library: SFML

Available from <https://www.sfml-dev.org/index.php> based on OpenGL

A basic game loop:

```
while (true)  
{  
    processEvents();  
    update();  
    render();  
}
```

OpenGL Window and Processing Events

```
#include <SFML/Window.hpp>
int main() {
    sf::Window window(sf::VideoMode(800, 600), "My window");
    while (window.isOpen()) {
        sf::Event event;
        while (window.pollEvent(event)) {
            if (event.type == sf::Event::Closed) window.close();
        }
        // draw everything here...
    }
    return 0;
}
```

Shapes and rendering

```
// clear the window with black color
window.clear(sf::Color::Black);
// draw everything here...
// e.g.
sf::CircleShape shape(50.f);
// set the shape color to green
shape.setFillColor(sf::Color(100, 250, 50));
window.draw(shape);
// end the current frame
window.display();
```

Positions

- Shapes (and sprites and text) are `sf::Transformable` and have
 - **position**,
 - **rotation**,
 - **scale**,
 - **origin**.

// set the absolute position of the entity
`entity.setPosition(10, 50);`

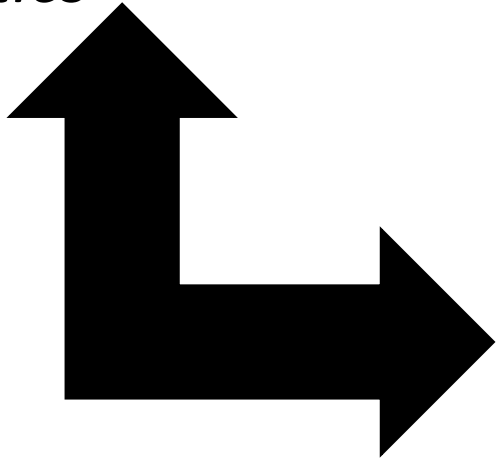
// move the entity relatively to its current position
`entity.move(5, 5);`

// retrieve the absolute position of the entity
`sf::Vector2f position = entity.getPosition();`

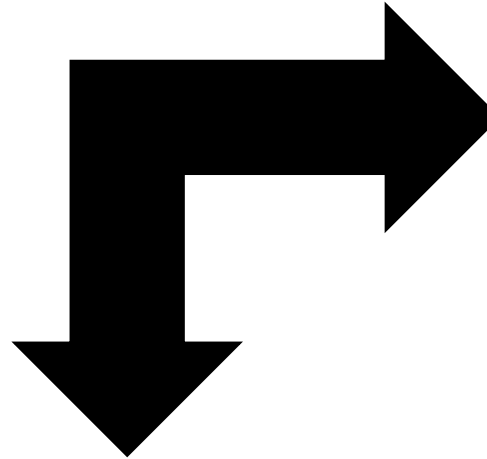
Positions

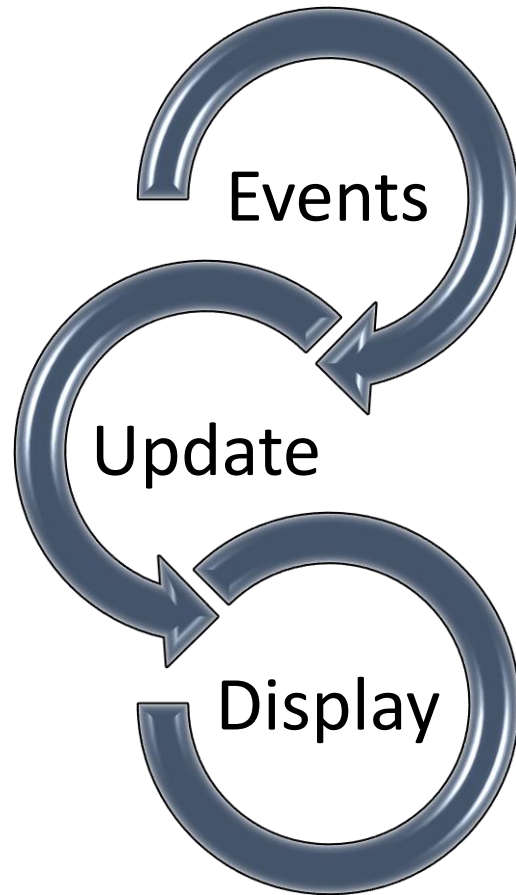
- Views, allows zoom or scroll
- Window resize changes cords, use `mapPixelToCoords` to convert
- Y coord is 0 at the top:

Mathematics



Graphics





```
while (true)
{
    processEvents();
    update(); // TODO
    render();
}
```

Swarm Intelligence

- Metaheuristics: For intractable optimization problems
 - If you are training a deep learning model with a stochastic gradient descent with backpropagation and cannot escape from the local minima
 - <https://towardsdatascience.com/swarm-intelligence-coding-and-visualising-particle-swarm-optimisation-in-python-253e1bd00772>
- ***“The emergent collective intelligence of groups of simple agents.”***
<https://www.swarm-intelligence.it/>
- Trajectory based
 - e.g. simulated annealing versus population based
- Agents usually move in two phases: exploration and exploitation

Particle swarm optimization: PSO

- Particles have a position (e.g. x, y) and a velocity (v_x, v_y)
- (x, y) are solutions to a problem
- They move according to

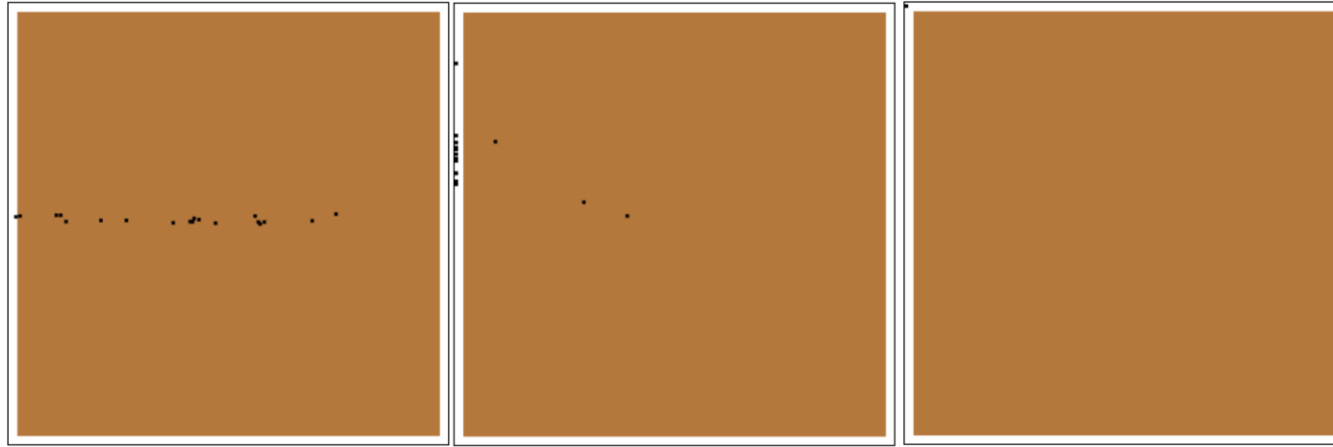
$$x_{t+1} = x_t + v_{x,t}$$
$$v_{x,t+1} = w \cdot v_{x,t} + c_1 \cdot r_1 \cdot (pb_t - x_t) + c_2 \cdot r_1 \cdot (gb_t - x_t)$$

Should say r_2

w – weight; c 's constants; r 's rnd numbers

pb , personal best; gb – global best

Particle swarm optimization: PSO



Choose **n**

Initialize **n** particles randomly

For a while:

- Update best global position

- Move particles

- Update each particle's best position & velocity

Ants

- Ants move on a grid/graph/network and find paths or trails.
- Ants consider places based
 - on pheromone, τ , laid on paths as ants move
 - and quality, η , a heuristic to encourage specific behaviour
 - using two parameters, α, β using

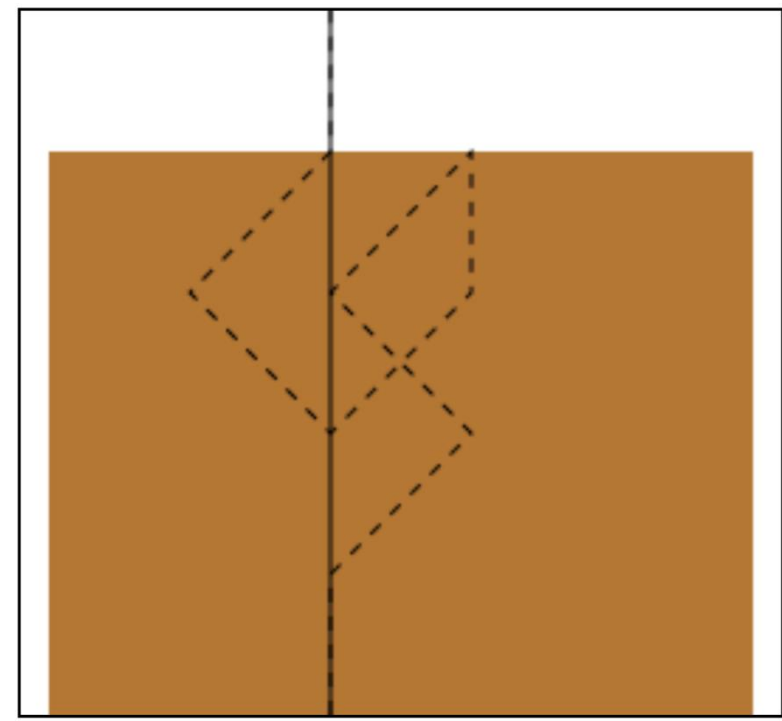
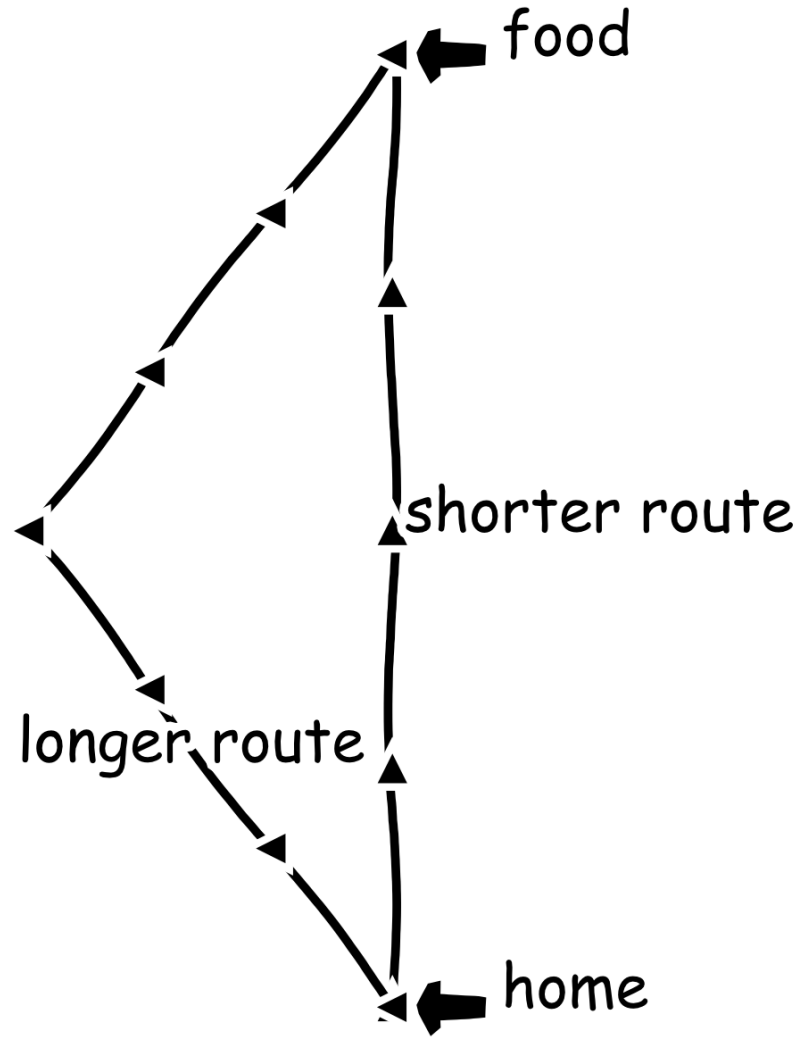
$$p(position)_i = \frac{\tau_i^\alpha \times \eta_i^\beta}{\sum_j \tau_j^\alpha \times \eta_j^\beta}$$

for a while:

create paths

update pheromones

evaporate and add to new paths



Abstract Bee Colony : ABC

Bees have one of three roles

1. Scout
2. Gather food
3. Stay at home/inactive

For a while:

Go out

Worker bees get food from a known source & explore nearby

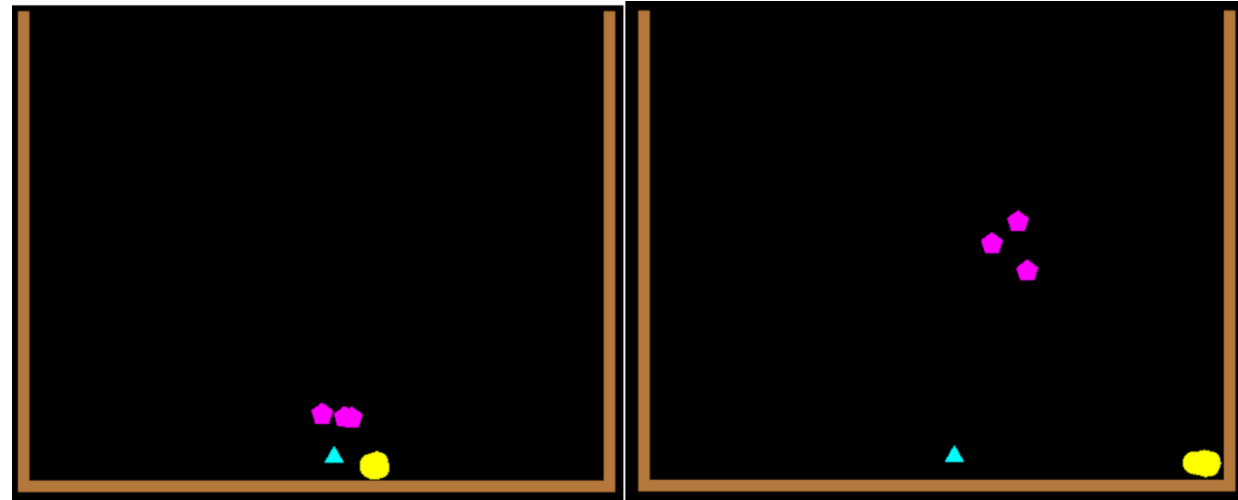
Inactive bees wait at home

Scout bees explore remembering the better food sources

Go Home

Waggle dance

recruit bees



Genetic Algorithms and Machine Learning for Programmers

Create AI Models
and Evolve Solutions



Frances Buontempo
edited by Tammy Coron

Time to let the cat(s) out of the bag



Cat swarm optimization (CSO)

- Chu et al. 2006
- Two modes
 1. **seeking**, looking nearby,
 2. **tracing**, heading to a target with a velocity
- See <https://www.sciencedirect.com/topics/computer-science/cat-swarm-optimization> for various articles
 - Workflow scheduling in cloud computing
 - Signal processing
 - Video compression, ...

- Step1: **Create N cats.**
- Step2: **Randomly sprinkle** the cats into the M -dimensional solution space and randomly select values, which are in-range of the maximum velocity, to the velocities of each cat. Then **haphazardly pick** number of cats and set them into tracing mode according to MR, and the others set into seeking mode.
- Step3: **Evaluate the fitness value of each cat** by applying the positions of cats into the fitness function, which represents the criteria of our goal, and keep the best cat into memory.
- Step4: **Move the cats** according to their flags, if cat k is in seeking mode, apply the cat to the seeking mode process, otherwise apply it to the tracing mode process.
- Step5: **Shuffle cat's modes.**
- Step6: Check the termination condition, if satisfied, terminate the program, and otherwise repeat step3 to step5.

... in fewer words

```
auto cats = random_cats();  
for a while()  
{  
    auto best = best_cat(cats);  
    for(auto & cat : cats)  
    {  
        cat.seek_or_trace(best);  
    }  
    shuffle(cats);  
}
```

```
struct Cat
{
    Position position{};
    Velocity velocity{0.0, 0.0};
    bool trace = false;
};
```



```
struct Position { double x{}; double y{}; };  
    //or array for > 2 dimensions
```

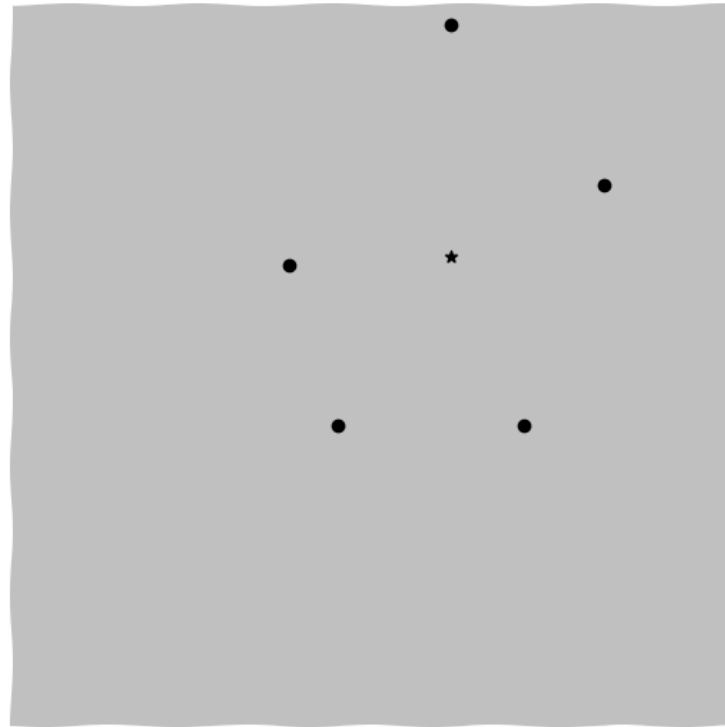
```
double angle(double x, double y);
```

```
class Velocity  
{  
    double x{}; double y{};  
    double theta{ angle(x, y) };  
public:  
    Velocity(double x, double y) : x(x), y(y) {}  
    double X() const { return x; }  
    double Y() const { return y; }  
    double Angle() const { return theta; }  
};
```

Seek

- Cat has
 - a memory seeking pool (SMP)
 - And may or may not consider its current position
 - a seeking range for selected dimension(s) (SRD)
- It seeks by
 - Making j copies of itself (or $j-1$ if considering current pos)
 - Add $\text{random}[-1, 1] * \text{SRD}$ to each
 - Add original pos if considering current pos
- Randomly choose one, but best more likely

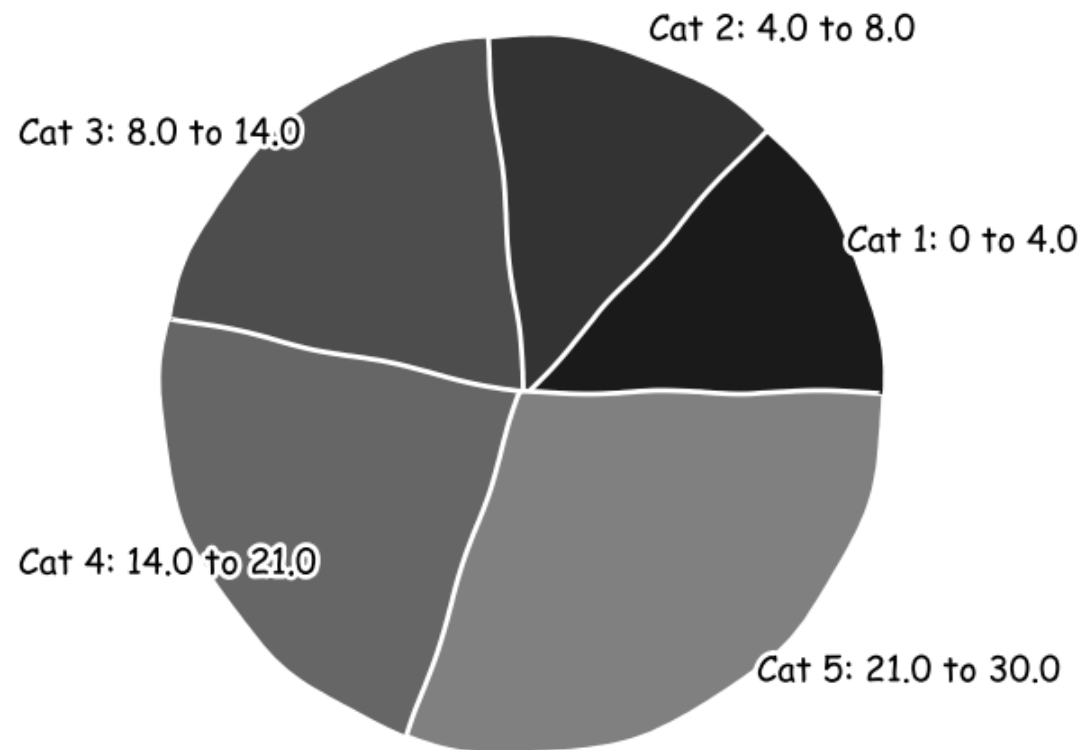
SMP of 5



$y = [4.0, 4.0, 6.0, 7.0, 9.0]$

SMP – roulette wheel selection

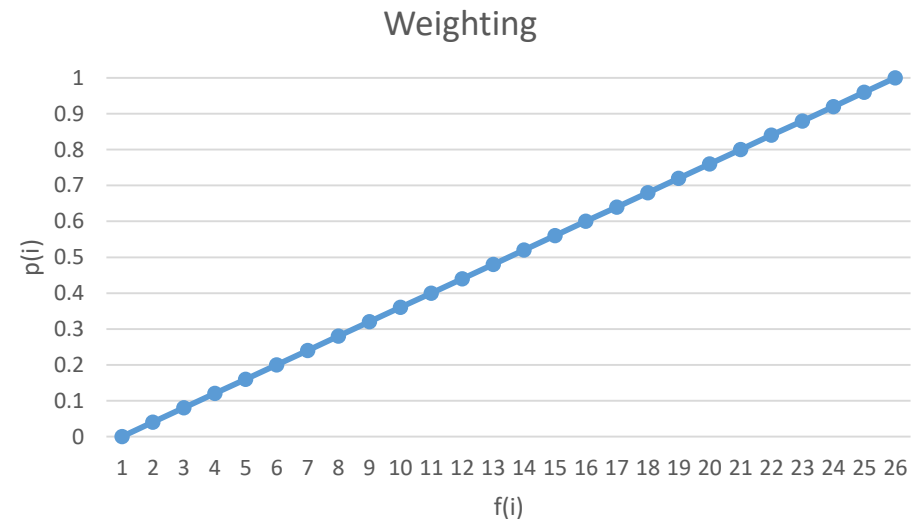
Warning: non-deterministic, which causes problems for regression testing, but we could seed or swap out the algo.



$y = [4.0, 4.0, 6.0, 7.0, 9.0]$

Seek choice

- $P_i = \frac{|F_i - F_b|}{F_{max} - F_{min}}$
- If the goal of the fitness function is to find the minimum solution, $F_b = F_{max}$, otherwise $F_b = F_{min}$.
- Notes
 - the worst has P of 0, so is never chosen
 - if $F_{max} = F_{min}$ all are equally likely

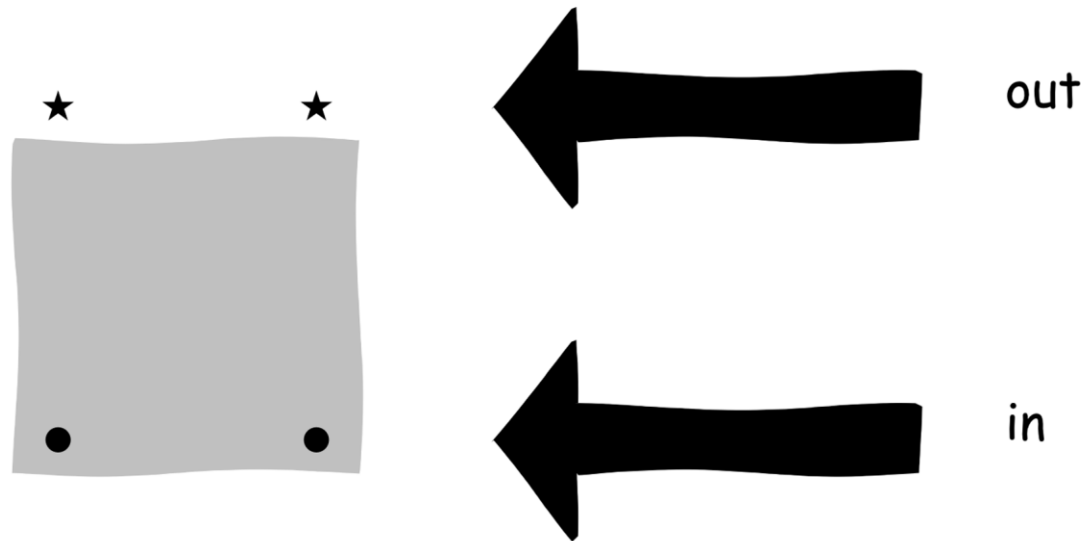


Objective, first version

```
class Objective {  
    std::function<double(double, double)> fn;  
public:  
    explicit  
    Objective(std::function<double(double, double)> fn);  
  
    double weighted_fitness(const Position& pos) const {  
        return fn(pos.x, pos.y);  
    }  
};
```

Let the cat out of the bag

```
auto fn = [](double x, double y) {  
    return std::max(0.0, y);  
};
```



```

std::discrete_distribution<int> fitness(const std::vector<Position>& positions,
                                       const Objective& objective) {
    std::vector<double> weights;
    std::transform(positions.cbegin(), positions.cend(), std::back_inserter(weights),
        [objective](const Position& p) { return objective.weighted_fitness(p); }
    );

    auto [smallest, biggest] = std::minmax_element(weights.cbegin(), weights.cend());
    if (*smallest == *biggest)
        return std::discrete_distribution<int>(weights.size(), 0, 1,
            [](auto x) { return 1.0; });

    double smallest_value = *smallest;
    std::transform(weights.begin(), weights.end(), weights.begin(),
        [smallest_value](auto w) { return w - smallest_value; }
    );
    return std::discrete_distribution(weights.begin(), weights.end());
}

```

Choice

```
Position choose_position_by_roulette_wheel(  
    const std::vector<Position>&positions,  
    const Objective& objective)  
{  
    static std::mt19937 gen(std::random_device{}());  
    auto distribution = fitness(positions,  
                                objective);  
    auto index = distribution(gen);  
    return positions[index];  
}
```

A static? Yikes

- Count how many times I do this
 - A prize for the first person to get it right
- Alternatives
 - Singleton, but see “Retiring the Singleton Pattern” by Peter Muldoon
 - Pass one in, but which one?
 - We have a named requirements: UniformRandomBitGenerator, which means templates
- Passing something in is easier to test
- A static in a function is IMPOSSIBLE to test
- A singleton is possible to avoid with a linker seam

Seeking (snoozing) cats

```
using Chooser = std::function<Position(const std::vector<Position>& positions,  
                                       const Objective& objective)>;  
  
class Seek {  
    int seeking_memory_pool{};  
    double seeking_range_of_the_selected_dimension{};  
    bool self_position_considering{};  
    Chooser choose_position;  
    std::mt19937 gen{std::random_device{}()};  
    std::uniform_real_distribution<> dist{ -1.0, 1.0 };  
public:  
    explicit Seek(int SMP, double SRD, bool SPC, Chooser chooser);  
    [[nodiscard]] Cat move(Cat pos, const Objective& objective);  
};
```

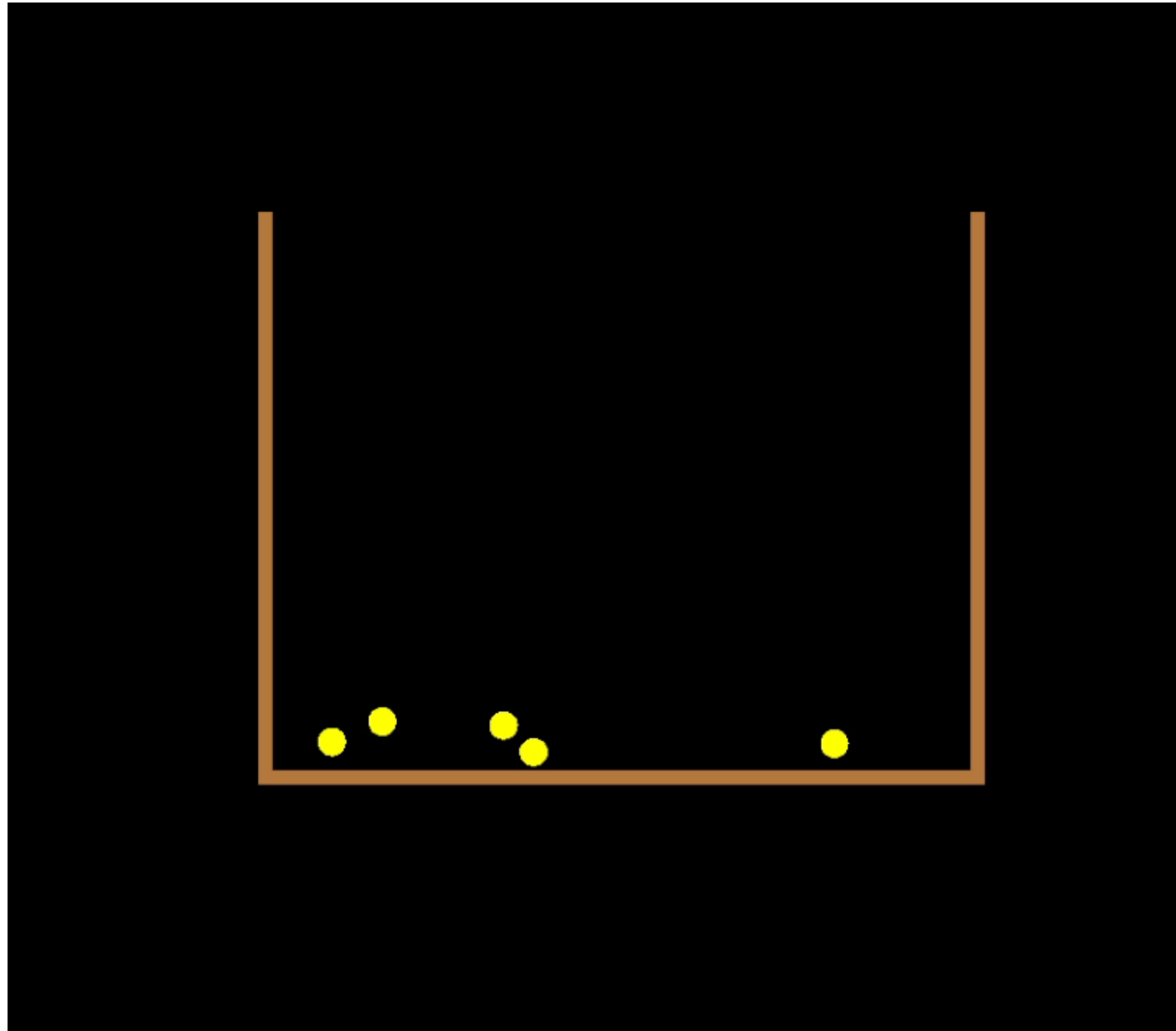
```

[[nodiscard]] Cat Seek::move(Cat cat, const Objective& objective) {
    auto potential_places = std::vector<Position>(seeking_memory_pool,
                                                    cat.position);

    auto move_fn = [this](auto& place) {
        auto percent = dist(gen);
        place.x += (percent * seeking_range_of_the_selected_dimension);
        percent = dist(gen);
        place.y += (percent * seeking_range_of_the_selected_dimension);
    };
    std::for_each(self_position_considering ? ++potential_places.begin() :
                potential_places.begin(),
                potential_places.end(),
                move_fn);

    cat.position = choose_position(potential_places, objective);
    cat.trace = false;
    return cat;
}

```



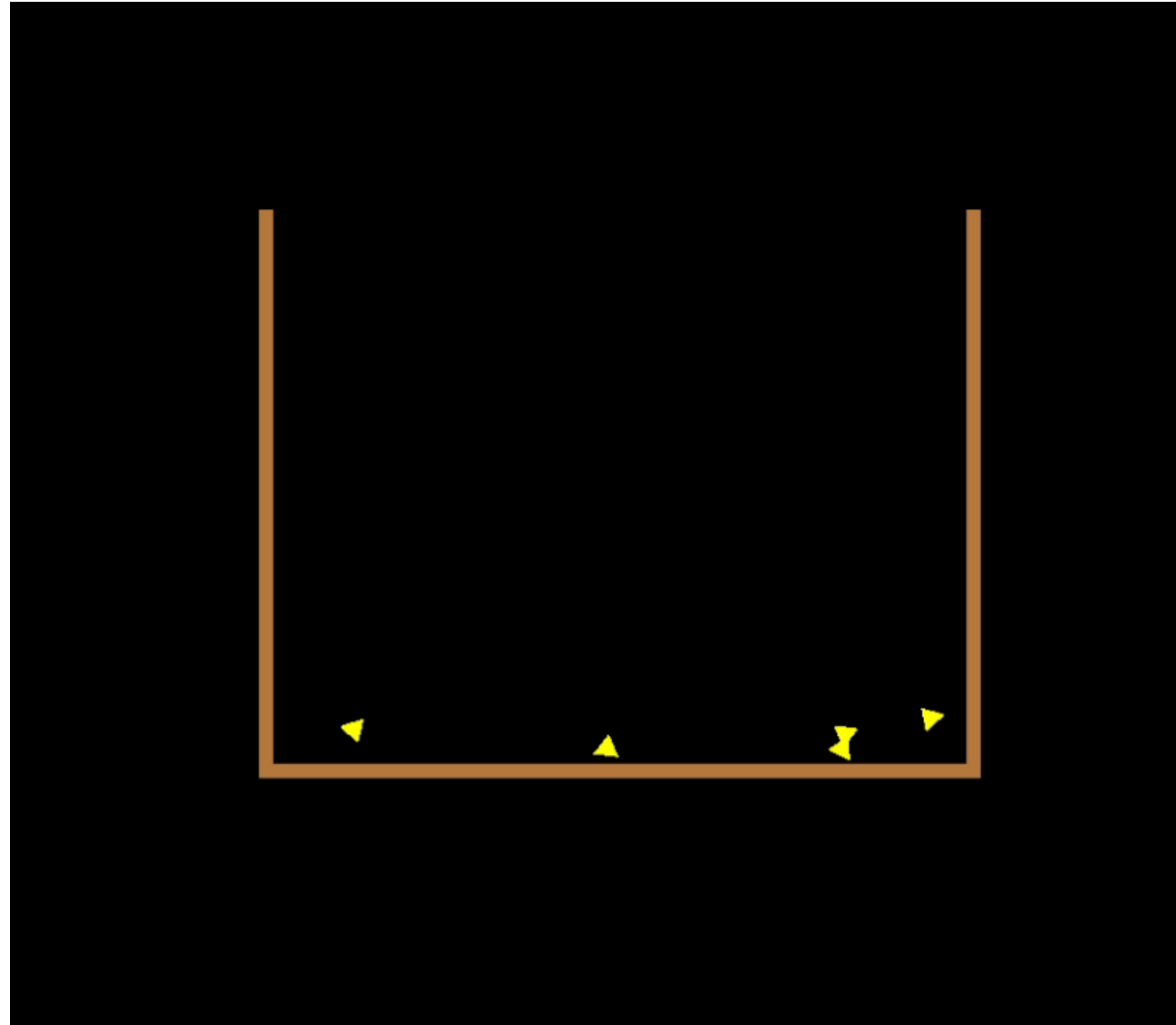
Trace

```
class Trace
{
    double c1{};
    double max_velocity{};
    std::mt19937 gen{ std::random_device{}() };
    std::uniform_real_distribution<> dist{ 0.0, 1.0 };
public:
    explicit Trace(double c1, double max_velocity)
        : c1(c1),
          max_velocity(max_velocity) {
    }

    [[nodiscard]] Cat move(Position best, Cat cat);
};
```

Hunting

```
[[nodiscard]] Cat Trace::move(Position best, Cat cat) {  
    auto r1 = dist(gen);  
    auto v = cat.velocity.X() + r1 * c1 * (best.x - cat.position.x);  
    auto v_x = std::clamp(v, -max_velocity, max_velocity);  
    r1 = dist(gen);  
    v = cat.velocity.Y() + r1 * c1 * (best.y - cat.position.y);  
    auto v_y = std::clamp(v, -max_velocity, max_velocity);  
    // Or move by v and update v elsewhere?+++++  
    // Vim added the +'s :-)  
    cat.velocity = Velocity{v_x, v_y};  
    cat.position.x += v_x;  
    cat.position.y += v_y;  
    cat.trace = true;  
    return cat;  
}
```



Both seeking and tracing

- Why both?
- How many of each?
 - Algo has a mixture ratio, but we can specify how many of each
- We'll use a CSO class to hold params, and do updates to the cats.

CSO class – member variables

```
std::vector<Cat> cats{};
double max_x{};
double max_y{};
double max_v{};
int tracing{ std::count_if(cats.begin(), cats.end(),
    [](const Cat & cat) { return cat.trace; }) };
double good{};

Trace trace{ 1.0, max_v };
Seek seek{ 5, 2.5, false };
bool swap_cat_modes{ true };

Objective objective;
Cat best{ best_position(cats, objective) };
std::mt19937 gen{ std::random_device{}() };
```

CSO – some useful functions

```
std::vector<Cat> Cats() const { return cats; }  
Cat Best() const { return best; }
```

```
double weighted_fitness(const Position & pos) const  
{  
    return objective.weighted_fitness(pos);  
}
```

```
bool good_kitty(const Cat & cat) const  
{  
    return weighted_fitness({ cat.position.x, cat.position.y })  
        >= good;  
}
```

```

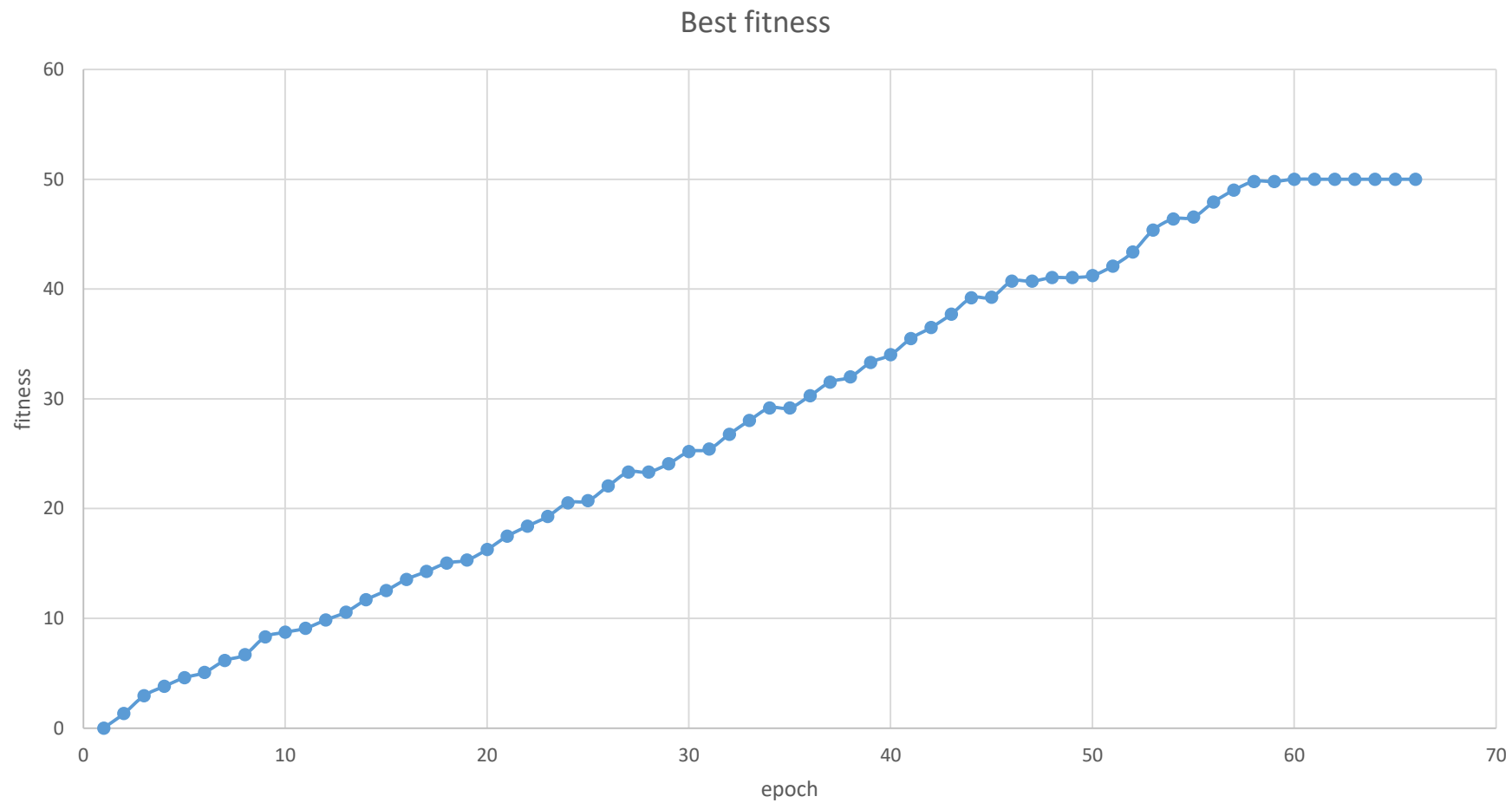
void CSO::Update() {
    if (swap_cat_modes)
        std::shuffle(cats.begin(), cats.end(), gen);

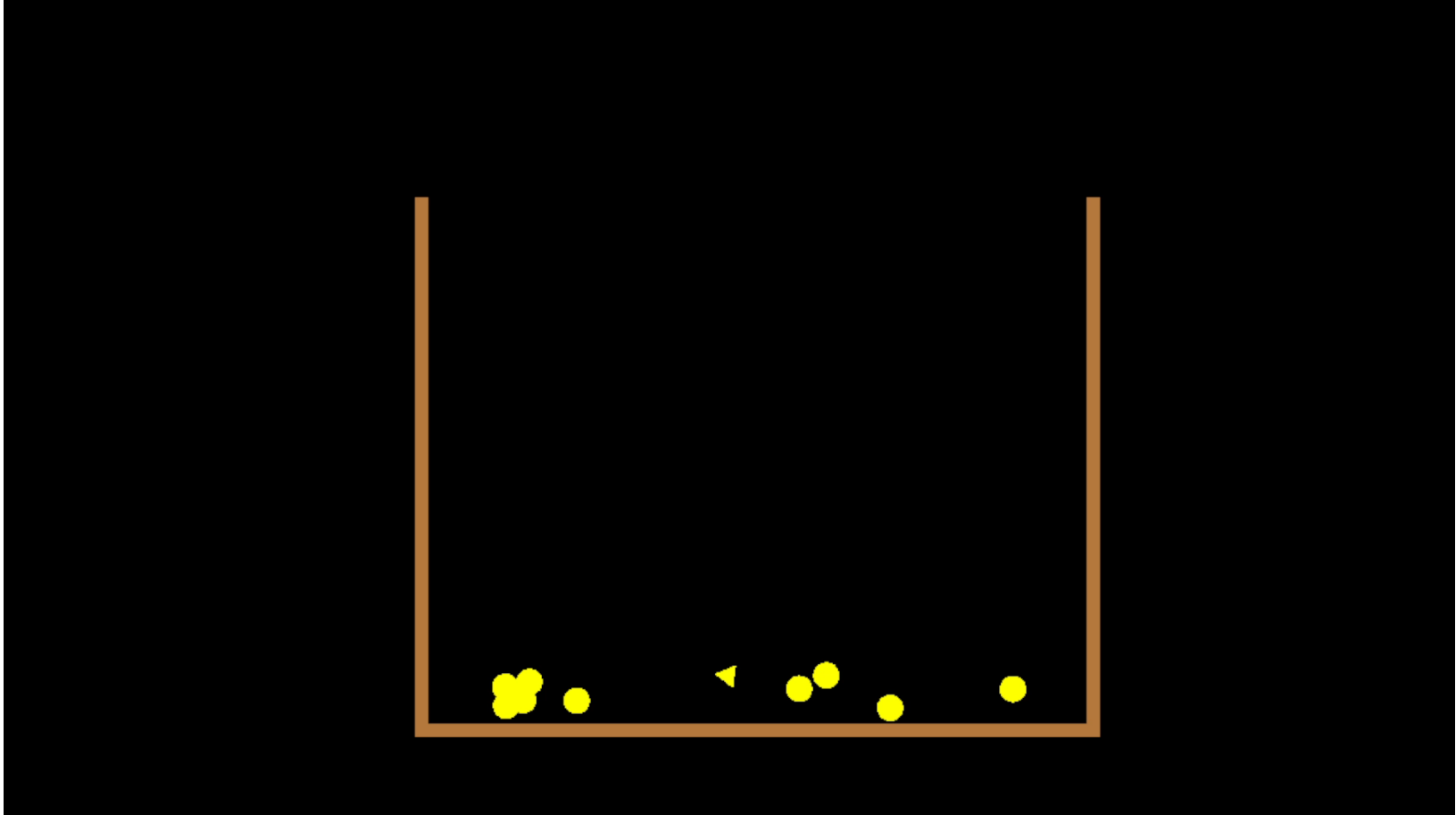
    for (int i = 0; i < tracing; ++i) {
        cats[i] = trace.move(best.position, cats[i]);
    }
    const auto number_of_cats = cats.size();
    for (size_t i = tracing; i < number_of_cats; ++i) {
        cats[i] = seek.move(cats[i], objective);
    }
    Clamp();

    auto current_best = best_position(cats, objective);
    if (weighted_fitness(best.position) <=
        weighted_fitness(current_best.position))
        best = current_best;
}

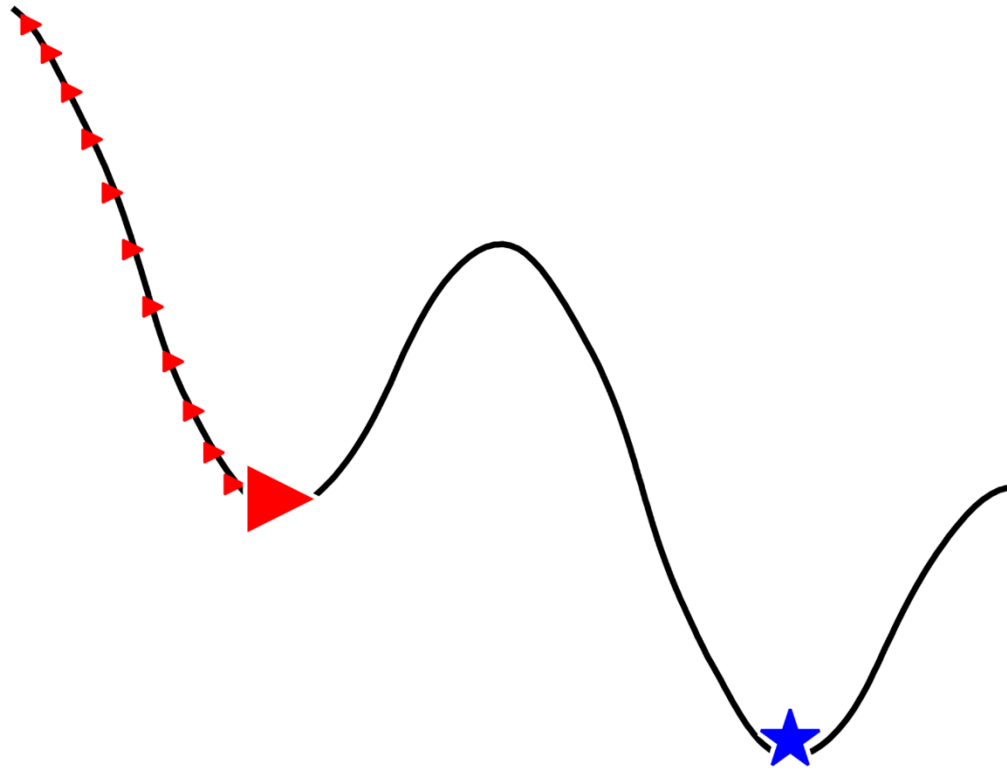
```

```
void Swarm::CSO::Clamp()
{
    for (auto & cat : cats) {
        cat.position.x =
            std::clamp(cat.position.x, 0.0, max_x);
        cat.position.y =
            std::clamp(cat.position.y, 0.0, max_y);
    }
}
```

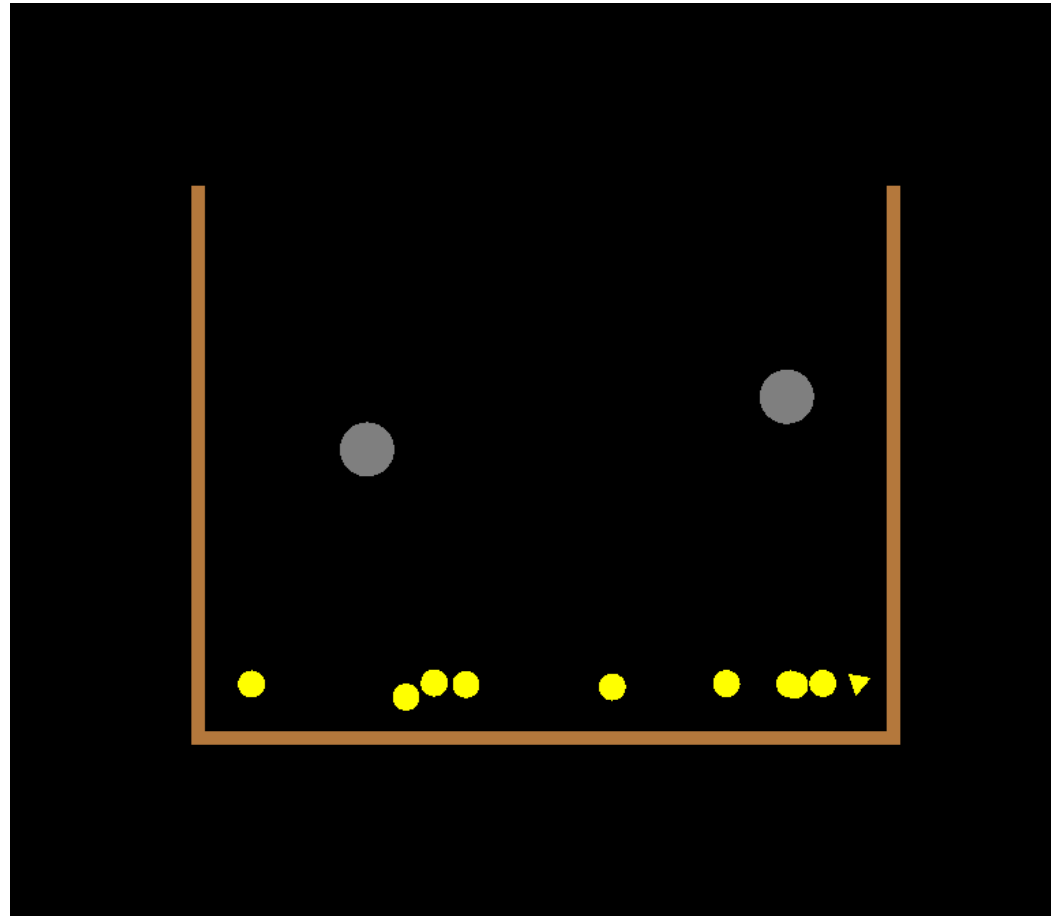




Local minima/maxima



Distracted cats



The Specials

```
std::vector<std::tuple<Swarm::Position, double>> Swarm::make_special_positions(double max_x, double max_y,
    int number_of_special_points, double weight)
{
    static std::mt19937 gen(std::random_device{}());
    std::vector<std::tuple<Position, double>> special;
    std::uniform_real_distribution<> special_x_dist(max_x * 0.1, max_x * 0.9);
    std::uniform_real_distribution<> special_y_dist(max_y / 8.0, max_y / 2.0);
    for (int i = 0; i < number_of_special_points; ++i)
    {
        double x = special_x_dist(gen);
        double y = special_y_dist(gen);
        special.push_back({ {x, y}, weight });
    }
    return special;
}
```

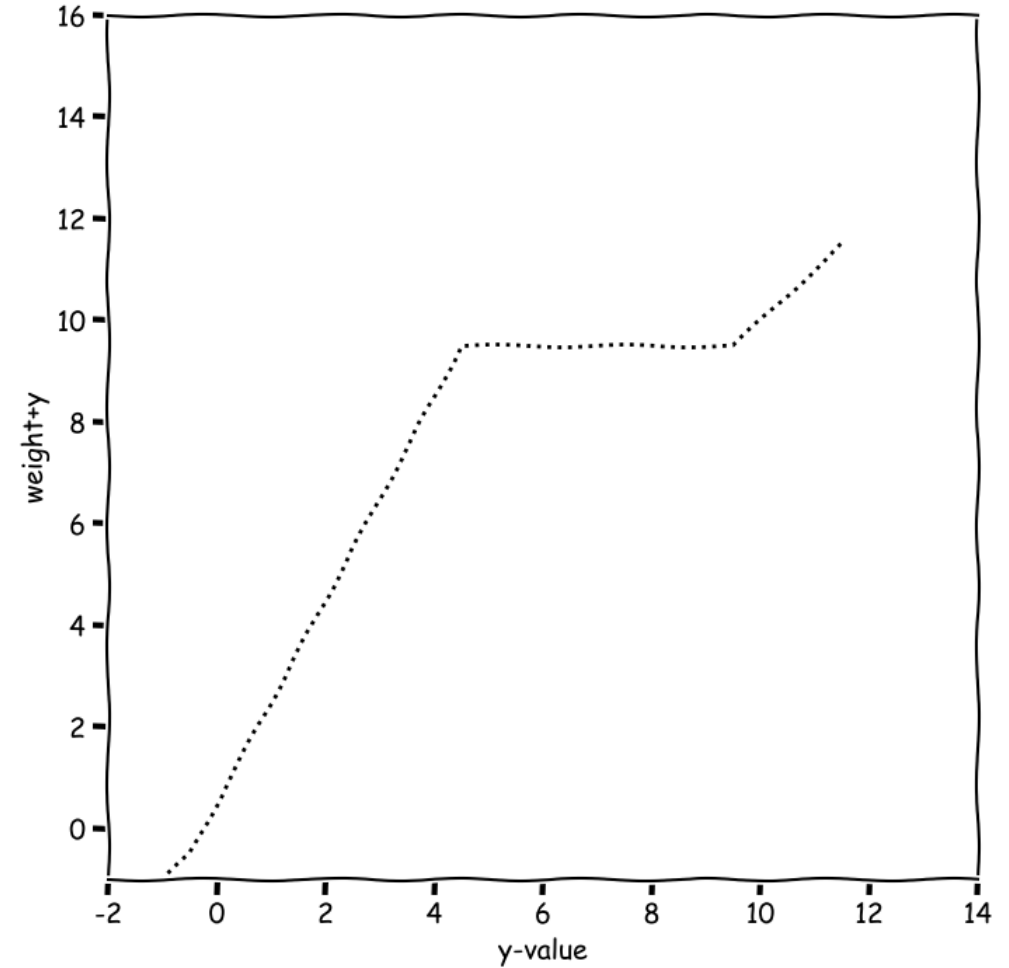
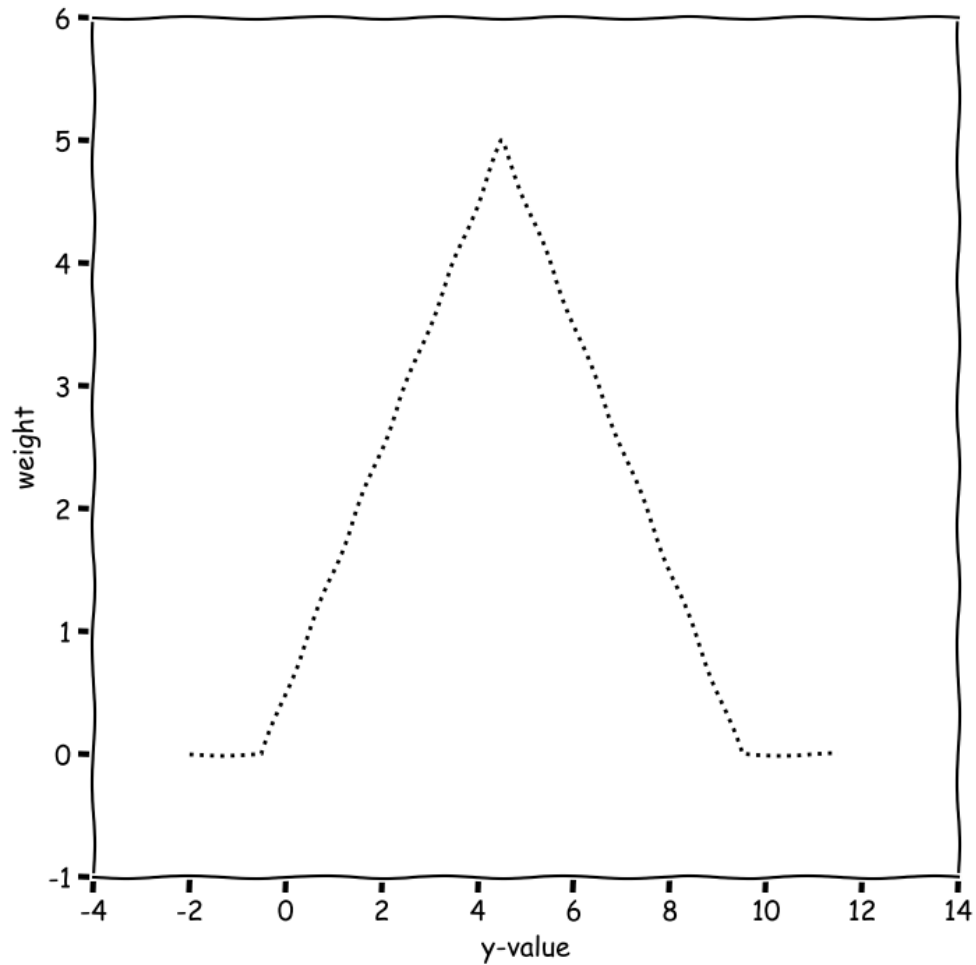
The Objective

```
class Objective
{
    std::vector<std::tuple<Position, double>> special{};
    std::function<double(double, double)> fn;
public:
    explicit Objective(const std::vector<std::tuple<Position, double>>& special,
                      std::function<double(double, double)> fn);

    double weighting(Position pos) const;
    double weighted_fitness(const Position& pos) const {
        return (fn(pos.x, pos.y) + weighting({ pos.x, pos.y }));
    }

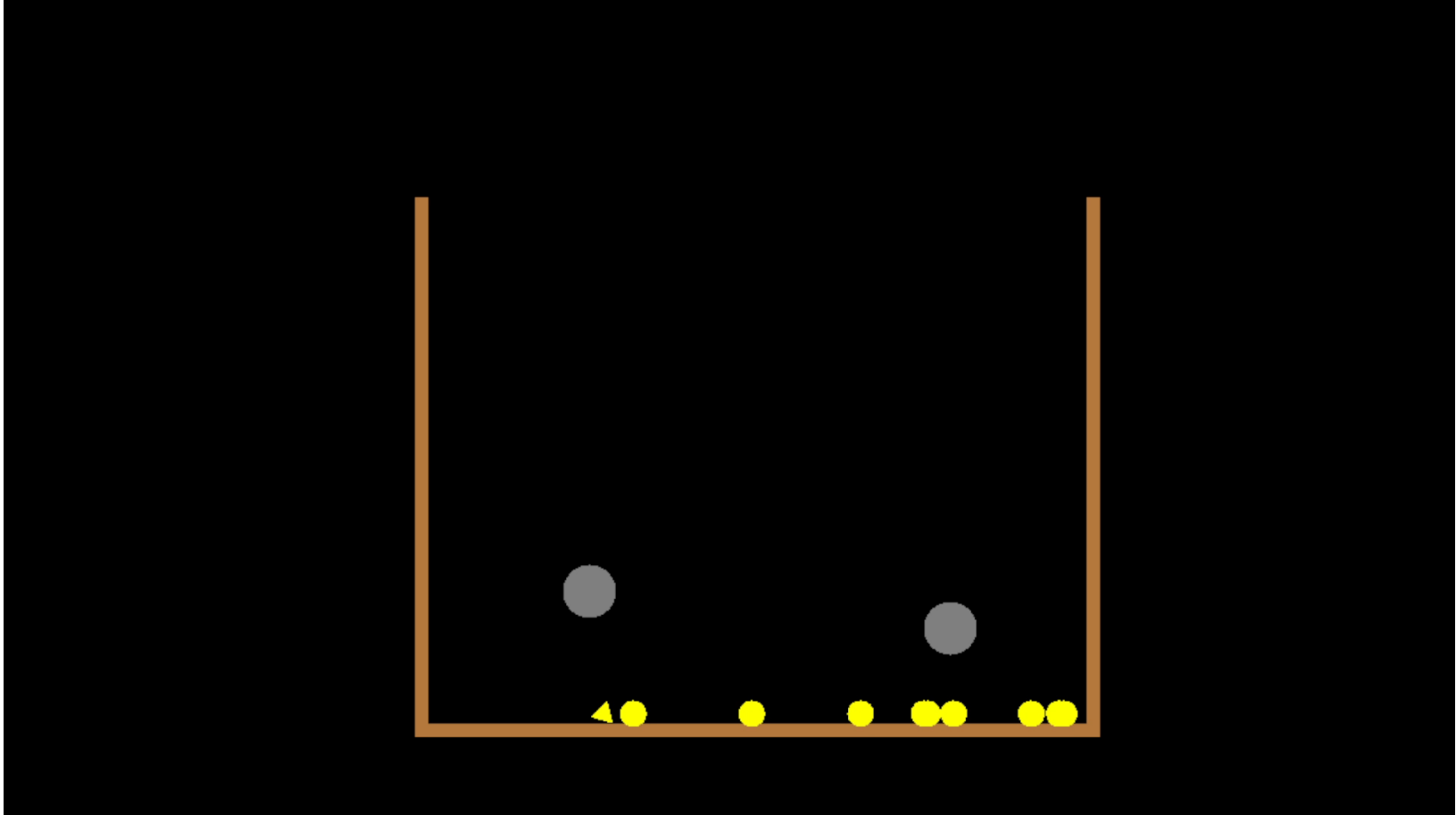
    std::vector<std::tuple<Position, double>> Specials() const { return special; }
};
```

Weighting...



Weighting

```
double Swarm::Objective::weighting(Position pos) const
{
    auto value = 0.0;
    for (const auto& [special_pos, weight] : special)
    {
        auto dist = distance(pos.x, pos.y, special_pos.x, special_pos.y);
        if (dist <= weight)
        {
            value += (weight - dist);
        }
    }
    return value;
}
```



Benchmarking: separability, multimodality, and regularity

“A function is multimodal if it has two or more local optima.

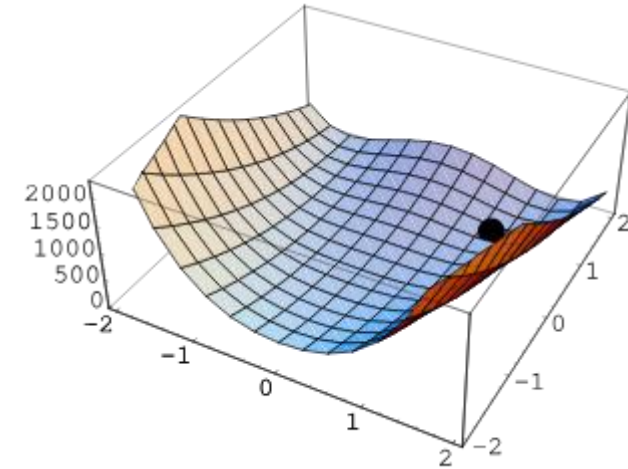
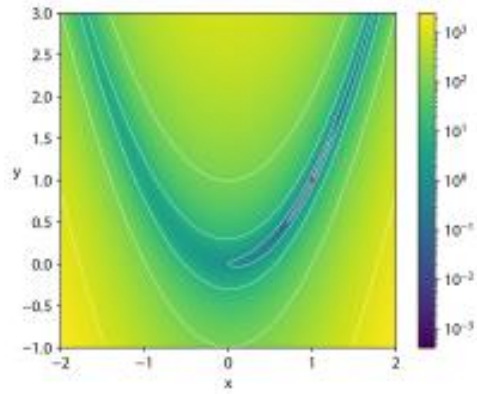
A function of p variables is separable if it can be rewritten as a sum of p functions of just one variable.”

A regular function is differential and a *function*

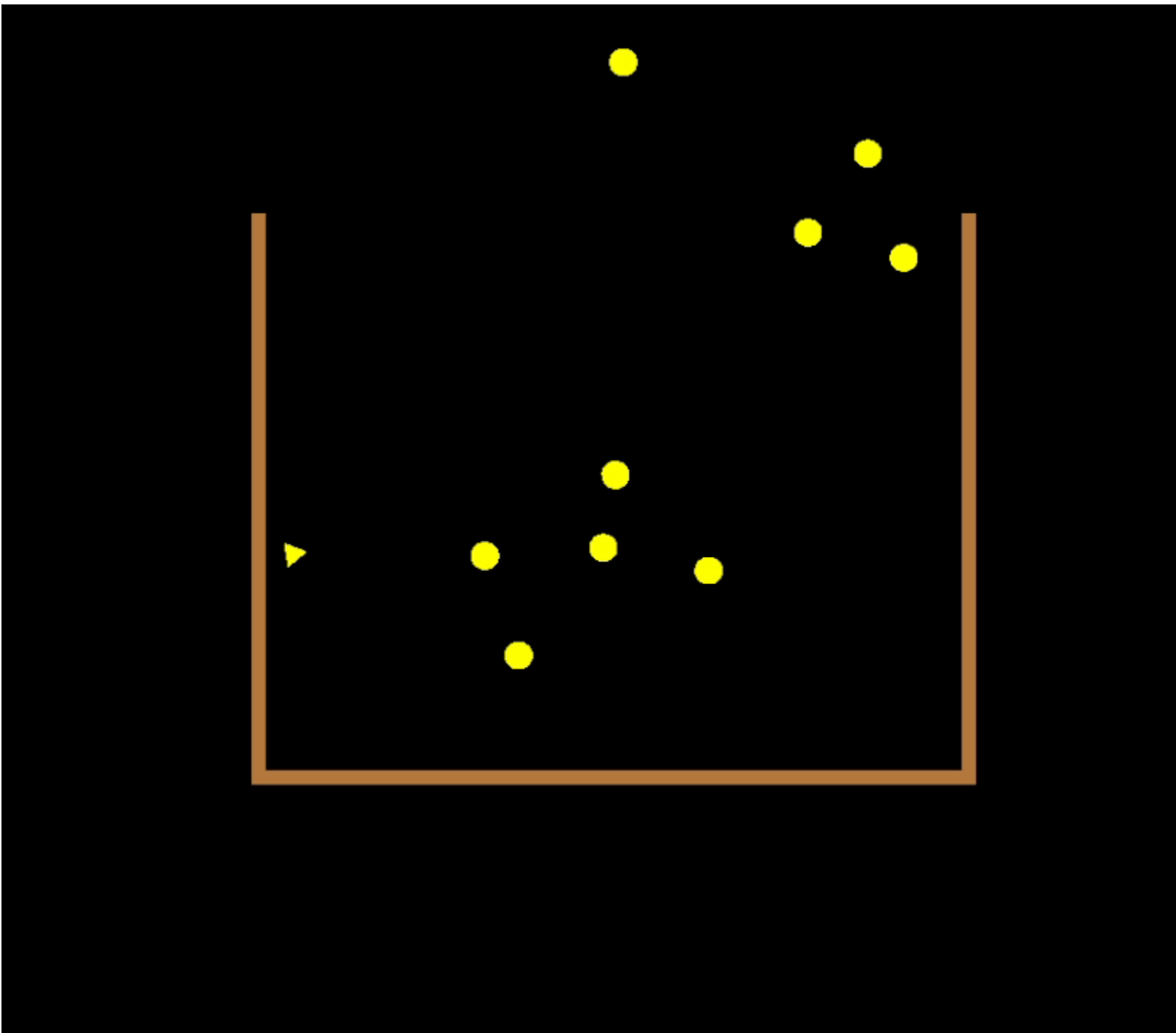
“The separability is closely related to the concept of epistasis or interrelation among the variables of the function. In the field of evolutionary computation, the epistasis measures how much the contribution of a gene to the fitness of the individual depends on the values of other genes.”

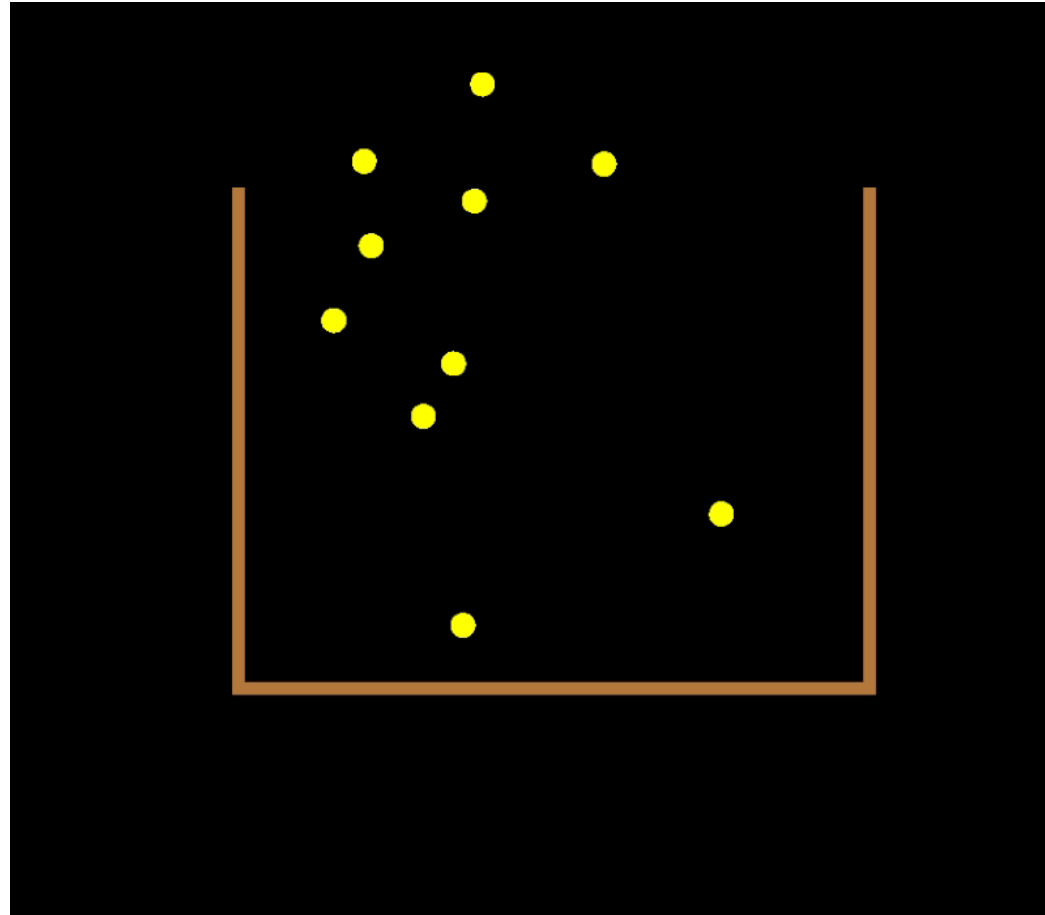
<https://www.cs.cmu.edu/afs/cs/project/jair/pub/volume24/ortizboyer05a-html/node6.html>

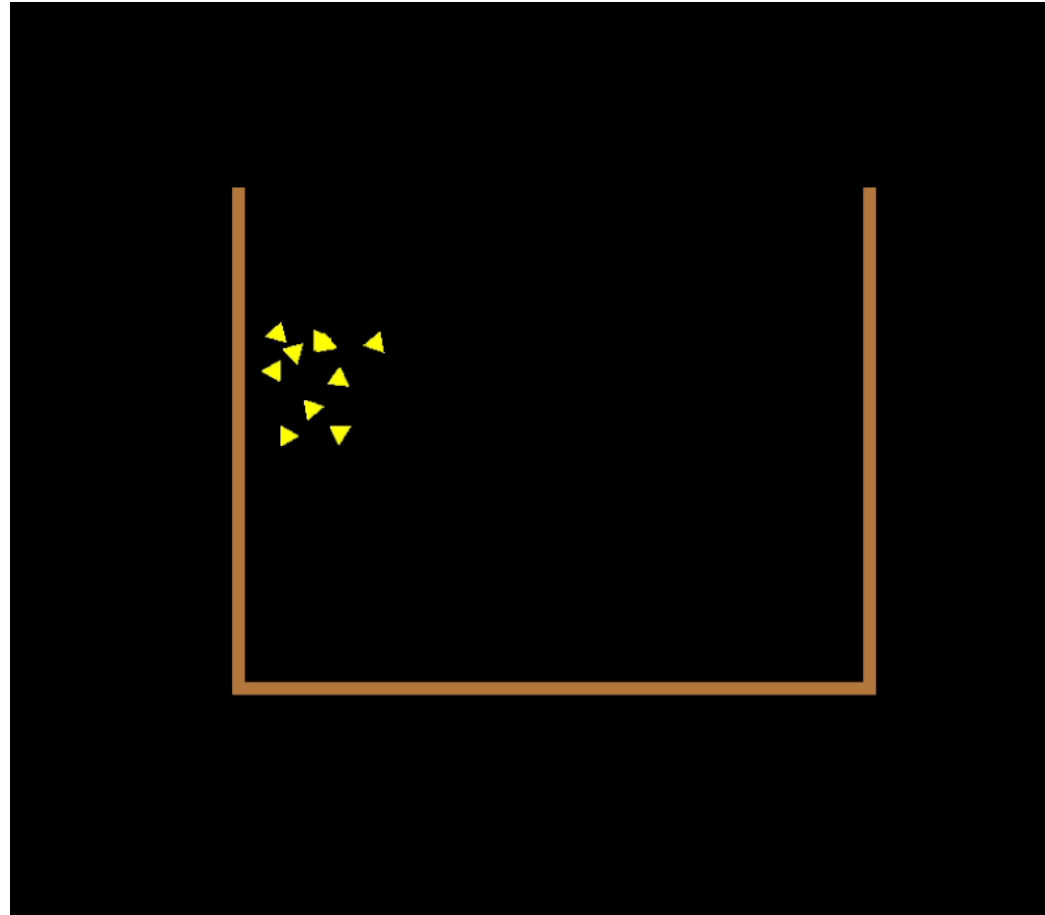
Rosenbrock



- $2D$
 - $f(x, y) = (x - a)^2 + b(y - x^2)^2$
 - With $a = 1$ and $b = 100$:
 - $f(x, y) = (x - a)^2 + b(y - x^2)^2$
- Can be extended to more dimensions
 - e.g. $f(x) = \sum_{i=1}^{p-1} \left[(x_i - 1)^2 + 100(x_{i+1} - x_i^2)^2 \right]$
- Also, others on https://en.wikipedia.org/wiki/Test_functions_for_optimization







Sum up

- Swarm algorithms have
 - position (solutions) and trajectories – velocities.
 - a for loop and some “randomness”
 - personal/global bests
 - various possible roles/modes
- Implementing them is a good way to practice coding
- They can solve many intractable problems
- Related to “artificial life” e.g. robots/software do tasks on their own.
 - Reynolds’ “Boids” <https://www.red3d.com/cwr/boids/>

Genetic Algorithms and Machine Learning for Programmers

Create AI Models
and Evolve Solutions



Use code **folkstone2024**
to get 35% off until the
end of July.



Frances Buontempo
edited by Tammy Coron

45% discount code **CPPONSEA45** (valid from Jun 30 to Sep 30)

<https://mng.bz/0GEp>

